

Engineering Reflection

1. Why should source profiling occur before implementation of ingestion?

- Building an ingestion pipeline without profiling from the source first is like writing an essay without reading the prompt. The assumptions based into the code end up being wrong in ways that are expensive to fix later. In this lab, `customer_id` appeared to be clean surrogate key until profiling showed that three ID's repeat and one of them maps to two completely different people. Had the pipeline been built first, that discovery would have required rewriting logic that had already run against real data. The time spent profiling upfront is always less than the time spent untangling a pipeline that was designed around incorrect assumptions.

2. What is the difference between source event time and ingestion time

- These two timestamps measure completely different things. Source event time, represented by `updated_at` in the API, reflects when a record was last modified in the originating system. That could have happened hours or weeks before the pipeline ever touched it. Ingestion time, stored as `_ingested_at`, is simply when the pipeline collected and wrote the record. The watermark tracks source event time rather than ingestion time because the goal is to capture the actual state of the data in the source, not to record when the pipeline happened to run. Using ingestion time for the watermark would cause records to be skipped or double-fetched depending on scheduling, which defeats the purpose of incremental ingestion entirely.

3. Why is `event_id` alone insufficient to decide which duplicate API record to keep?

- An `event_id` identifies which event is being described but says nothing about which version of that event reflects the current state. The API in this lab deliberately assigns the same `event_id` to records with different `updated_at` values to test exactly this. Relying on `event_id` alone would mean picking one of the two versions arbitrarily, with no guarantee that the chosen record is the most recent. The correct approach is to use both fields together: group by `event_id` and keep the record with the greatest `updated_at`, which is the only way to ensure the deduplicated output reflects the latest known state of each event.

4. Why must watermark state advance only after successful persistence?

- The ordering here is the entire safety guarantee of the watermark pattern. If the watermark is written before the raw file is confirmed as successfully written, any failure between those two steps leaves the pipeline in a state where it believes it has already processed records it has not actually stored. The next run requests only records newer than the advanced watermark, so the lost records are never fetched again. There is no error, no warning, just a quiet and permanent gap in the data. Advancing the watermark only after a confirmed write means a failure at any point leaves the pipeline in a recoverable state where it can safely retry the same window.

5. What limitation does `updated_after` greater than watermark have when multiple source records share exactly the same timestamp?

- The strict greater than comparison treats the watermark as an exclusive lower bound. Any record sitting exactly at the watermark value is excluded from the next run regardless of whether it was successfully ingested before. If a batch of records all share the same `updated_at` as the current watermark and some were not captured in the previous run, those records are silently skipped on every future run because they will never satisfy the strictly greater than condition. The pipeline has no mechanism to detect this since it only persists the watermark value, not the set of records ingested at that exact timestamp. This is a real operational risk for sources that assign timestamps in bulk.

6. How is duplicate prevention related to idempotency?

- Idempotency is the property that running the pipeline multiple times produces the same result as running it once. Duplicate prevention is the mechanism that makes that property hold in practice. Without the SHA-256 hash check for files, every rerun would write another copy of the same data into `raw/files`. Without `event_id` deduplication for the API, every rerun that overlaps with a previous window would append additional copies of the same records. In both cases the raw output grows with every run and the results diverge from what a single run would have produced. Duplicate prevention is what keeps reruns from being visible in the output.

7. Why should the raw area preserve source values instead of applying business transformations?

- The raw area serves as the system of record for what the source actually delivered. Once a transformation is applied there, the original values are gone and any error in the transformation logic has no correction path that does not involve re-ingesting from the source. Keeping raw values unchanged means the transformation layer can always be

rebuilt from scratch without touching the source again. It also isolates ingestion failures from transformation failures, which makes debugging significantly easier since the two layers can be reasoned about independently. Different consumers can also derive different views from the same raw data without any of them affecting what the others receive.

8. How could querying a production OLTP source for profiling or extraction degrade the application?

- Production OLTP systems are optimised for high-concurrency, low-latency, row-level transactions. Analytical queries sit at the opposite end of that spectrum. A full table scan competes directly with live application traffic for CPU, memory, and I/O, and depending on the database and isolation level it can also introduce lock contention that slows down or blocks writes from the application. Even a bounded query like `SELECT COUNT(*)` against a large unindexed table can cause noticeable degradation for active users. This is why the Task 1.6 queries used `LIMIT` and why the instructions explicitly prohibited cross joins and full-table analytical workloads against the source database.

9. What would change if the API had a rate limit of 60 requests per minute?

- The pagination loop would need to be throttled to avoid exceeding the limit. At 60 requests per minute the ceiling is one request per second, so introducing a one second sleep after each page fetch would keep the pipeline within bounds. A more production-appropriate approach would track the number of requests made within the current minute and only introduce a delay when approaching the limit, rather than sleeping unconditionally after every page. This avoids penalising fast API responses unnecessarily while still guaranteeing the rate limit is never breached.

10. How would extending this pipeline from a local raw area to PostgreSQL preserve rerun safety?

- The structural logic of the pipeline would remain the same but the persistence layer changes. File writes are replaced by database transactions and the atomic rename pattern used for the JSONL output maps directly to PostgreSQL transaction commit semantics. The SHA-256 manifest check for files would be replaced by an existence check against the target table before inserting. For API events, the `event_id` deduplication logic stays unchanged but the output goes into a table using an upsert rather than a plain insert, so reruns update existing records rather than creating additional copies. Critically, the watermark would still only advance after the transaction commits, which preserves the same safety guarantee that makes the file-based pipeline recoverable from failure.

